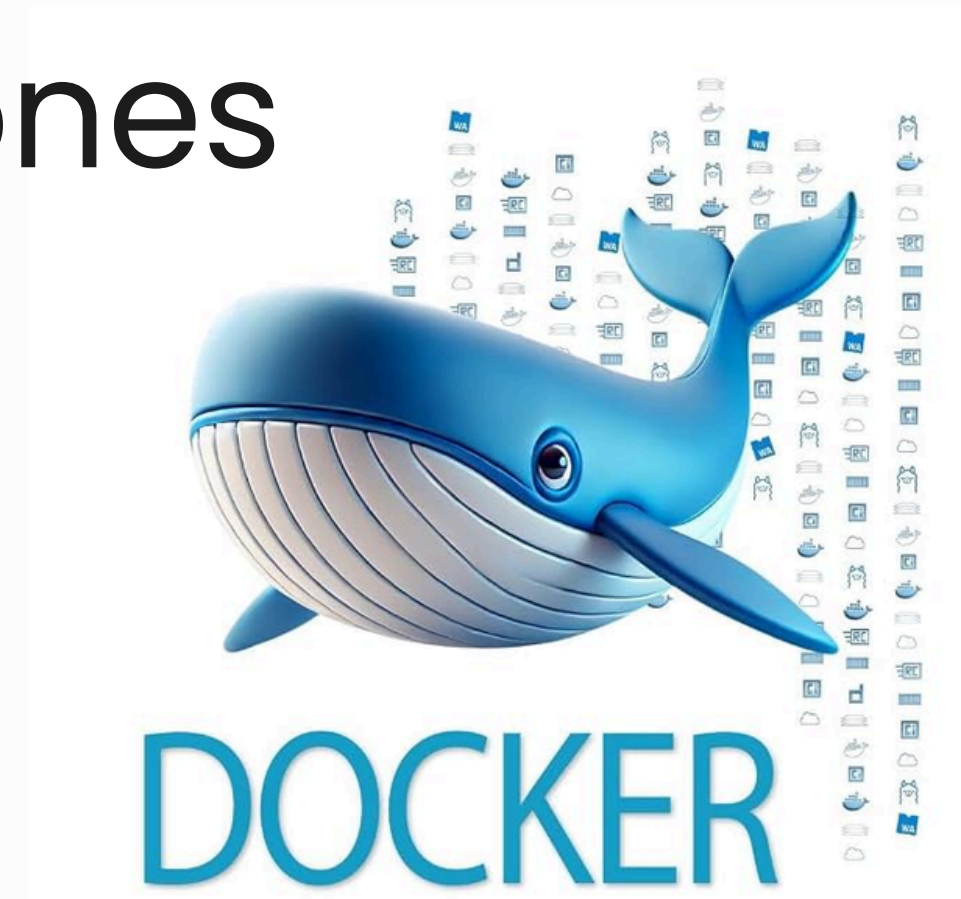


DOCKER DESDE CERO:

Crea y Despliega Aplicaciones

INSTRUCTOR: Cristian Jampier Chileno Segundo
Oficina de Tecnologías de la Información (OTI)
Programa Completo — PIT 2026



RESUMEN DE LA SESIÓN 2

1. Un Dockerfile define la receta para construir imágenes propias.
2. FROM, RUN, COPY, ENV, EXPOSE y CMD cubren el flujo base de una app.
3. El orden de capas afecta directamente la velocidad del build.
4. .dockerignore evita enviar basura o secretos al contexto de build.
5. Slim y Alpine ayudan a reducir tamaño, con criterios distintos.
6. Docker Hub permite compartir imágenes mediante tags versionados.

Ahora: Sesión 3

Docker Compose: levantar aplicaciones multi-contenedor con Flask, PostgreSQL y variables de entorno.

SESIÓN 3

- 1.Orquestación Local
- 2.Estructura del docker-compose.yml
- 3.Comandos de Compose
- 4.Integración Flask + PostgreSQL
- 5.Red Interna y Servicios
- 6.Variables de Entorno

Objetivo de la sesión 3

Al terminar la sesión 3 podrás

- Explicar para qué sirve Docker Compose.
- Levantar proyectos reales con múltiples servicios.
- Entender la estructura de docker-compose.yml.
- Conectar una aplicación Flask con PostgreSQL.
- Configurar variables de entorno usando archivo .env.
- Ejecutar, revisar y detener un stack completo con Compose.



Un comando, varios servicios

Docker Compose

El problema: una app ya no vive sola

- Una app real suele necesitar base de datos, cache, proxy o workers.
- Ejecutar cada contenedor a mano vuelve el laboratorio repetitivo.
- Los comandos docker run crecen con puertos, redes, volúmenes y variables.
- El equipo necesita una forma reproducible de levantar todo el stack.

Pregunta clave

¿Cómo describimos una aplicación completa, no solo un contenedor?



QUÉ ES DOCKER COMPOSE



Definición práctica

Docker Compose permite definir y ejecutar aplicaciones multi-contenedor usando un archivo YAML versionable.

DE DOCKER RUN A DOCKER COMPOSE

Sin Compose

```
docker network create app-net
docker run -d --name db \
  --network app-net postgres:16
docker run -d --name web \
  --network app-net -p 5000:5000 \
  mi-flask:v3
```

Con Compose

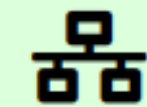
```
docker compose up -d
docker compose ps
docker compose down
```

Estructura de compose.yml

PIEZAS PRINCIPALES DE COMPOSE



services
Contenedores
del proyecto



networks
Comunicación interna



volumes
Datos persistentes



depends_on
Orden de arranque

Idea mental

Compose describe la topología local: qué servicios existen, cómo se conectan y qué datos conservan.

DOCKER-COMPOSE.YML MÍNIMO

3. Publicar la imagen

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    depends_on:
      - db
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: apppass
```

Lectura rápida

- web construye la app. db usa PostgreSQL.
- Compose crea una red interna por defecto.

COMANDOS ESENCIALES DE COMPOSE

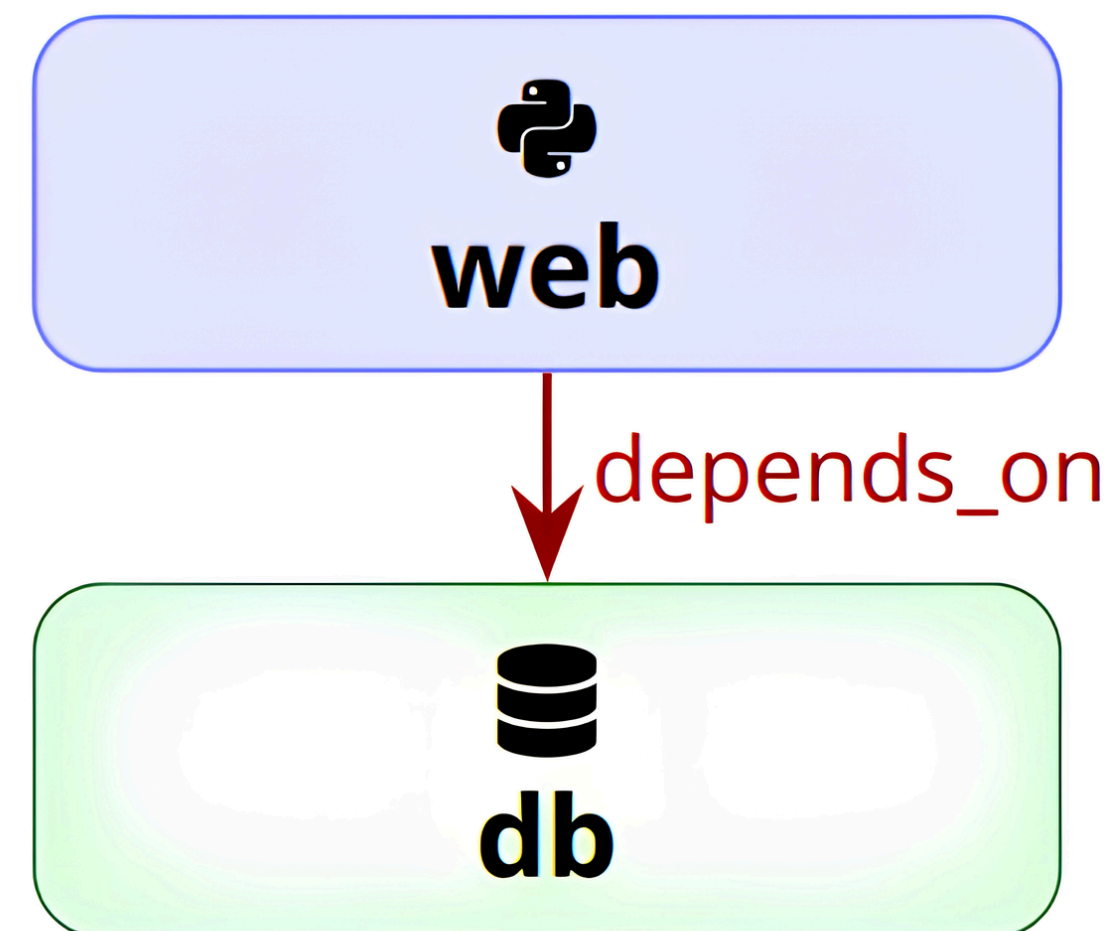
COMANDO	PARA QUÉ SIRVE	SOLUCIÓN RÁPIDA
<code>docker compose up -d</code>	Levanta el stack en segundo plano	Iniciar laboratorio
<code>docker compose ps</code>	Lista servicios del proyecto	Ver estado
<code>docker compose logs -f</code>	Muestra logs integrados	Diagnóstico
<code>docker compose exec web sh</code>	Abre shell en un servicio	Debug
<code>docker compose down</code>	Detiene y elimina contenedores/red	Limpiar stack

Dependencias entre servicios

- depends_on define orden de arranque básico.
- No garantiza que la base de datos ya acepte conexiones.
- Para producción o laboratorios robustos se usan healthchecks.
- En esta sesión nos enfocamos en levantar y conectar el stack.

Regla práctica

Arrancar antes no significa estar listo para recibir tráfico.



Flask + PostgreSQL

ARQUITECTURA DEL LABORATORIO



Clave de red interna

Dentro de Compose, la app no se conecta a localhost; se conecta al nombre del servicio: db.

VARIABLES DE CONEXIÓN EN FLASK

```
1 import os
2
3 DB_HOST = os . getenv ( " DB_HOST ", "db")
4 DB_NAME = os . getenv ( " DB_NAME ", " appdb ")
5 DB_USER = os . getenv ( " DB_USER ", " appuser ")
6 DB_PASSWORD = os . getenv ( " DB_PASSWORD ")
```

Por qué usar variables

- Evitan hardcodear credenciales.
- Permiten cambiar entorno sin tocar código.
- Funcionan bien con .env y Compose.

COMPOSE PARA FLASK + POSTGRESQL

services:

web:

build: .

ports:

- "5000:5000"

depends_on:

- db

db:

image: postgres:16

environment:

POSTGRES_DB: \${DB_NAME}

POSTGRES_USER: \${DB_USER}

POSTGRES_PASSWORD: \${DB_PASSWORD}

volumes:

- postgres_data:/var/lib/postgresql/data

volumes:

postgres_data:

Variables de entorno

ARCHIVO .ENV DEL PROYECTO

```
DB_HOST=db DB_NAME=appdb  
DB_USER=appuser  
DB_PASSWORD=appsecret  
FLASK_ENV=development
```

Importante

- No subas .env con credenciales reales a GitHub. Usa .env.example para documentar variables esperadas.

Para clase

- Usaremos valores simples de laboratorio para entender el flujo.

.ENV VS ENVIRONMENT

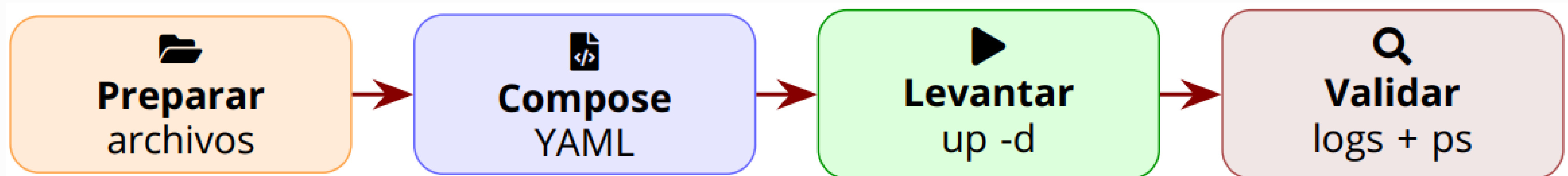
OPCIÓN	VENTAJA	CUÁNDO USARLA
environment	Visible en el YAML	Valores simples o explícitos
env_file	Separa configuración	Variables por entorno
.env.example	Documenta sin secretos	Repos compartidos
Secret manager	Mejor seguridad	Producción real

Regla práctica

El archivo .env ayuda en desarrollo; no es una bóveda de secretos.

Laboratorio multi-contenedor

FLUJO DEL LABORATORIO



Meta

Ejecutar una aplicación Flask conectada a PostgreSQL usando un solo archivo Compose.

COMANDOS DEL LABORATORIO COMPOSE

1. Levantar el stack

```
docker compose up -d --build
```

2. Revisar servicios

```
docker compose ps
```

3. Ver logs integrados

```
docker compose logs -f
```

4. Probar la app

```
curl http://localhost:5000
```

5. Entrar al servicio web

```
docker compose exec web sh
```

6. Detener y limpiar

```
docker compose down
```


ERRORES FRECUENTES EN COMPOSE

ERROR	CAUSA PROBABLE	SOLUCIÓN RÁPIDA
App no conecta a BD	Usa localhost en vez de db	Cambiar host al nombre del servicio
Puerto ocupado	Otro proceso usa 5000	Cambiar a 5001:5000
Variables vacías	Falta .env o nombre incorrecto	Revisar env_file y claves
BD pierde datos	No hay volumen persistente	Agregar volumen nombrado
Cambios no aparecen	Imagen antigua en caché	Usar up -d --build

Regla práctica

El archivo .env ayuda en desarrollo; no es una bóveda de secretos.

CHECKLIST DE APRENDIZAJE — SESIÓN 3

1. Puedo explicar para qué sirve Docker Compose.
2. Puedo leer la estructura básica de docker-compose.yml.
3. Puedo definir servicios, puertos, dependencias y volúmenes.
4. Puedo conectar Flask con PostgreSQL usando el nombre del servicio.
5. Puedo configurar variables con .env y env_file.
6. Puedo levantar y detener un stack con docker compose up y down.